

README – Assessment-1

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Contents of this folder

File	Description
Problem.pdf	The assignment statement (12 tasks, 50 marks).
Solution.pdf	Full written solution to Tasks 1–12 with derivations, tables and graphs.
Python_Code.pdf	Complete listing of student_search.py (algorithms, tests, experiment driver).
Output.pdf	Console-style results and the five generated plots.
Report.pdf	Short summary report of method, results and recommendation.
README.pdf	This file.

Algorithms implemented

A. Linear search (iterative). B. Binary search on sorted data (recursive; preprocessing = sort).
C. Hash table with separate chaining (iterative; preprocessing = build table).

How to run

```
python student_search.py
```

Requires Python 3 only (standard library). Runs in about one minute and writes results.json. Seeds are fixed (42, per-size seeds, 2026), so the same datasets and queries are used on every run; timings vary slightly by machine.

Key findings

At $n = 1,000,000$: linear search ≈ 22.6 ms per search, binary ≈ 2.44 μ s, hash ≈ 0.21 μ s.
Recommended design: sort + binary search for the original problem; hash index rebuilt daily (sorted array retained) for 50,000 searches/hour with once-a-day updates.